



计算机基础知识

第一章 计算机中的信息表示 (数据信息的表示部分)

2025-8-22



信息与软件工程学院
School of Information and Software Engineering



主要内容

- 1 数值型数据的表示方法
- 2 字符的表示方法

重点：数值型数据的表示

1、进位计数制及其相互转换

(二--八--十--十六进制间的转换)

2、机器数

3、IEEE754标准浮点表示格式

计算机是处理信息的工具，那么信息在其中是如何表示的呢？



1.1 数值型数据的表示方法



一个数值型数据的完整表示需三个方面：

进位计数制、符号的数字化（机器数）、小数点的处理

➤ 01. 进位计数制及其相互转换

➤ 02. 带符号数的表示

➤ 03. 数的定点表示和浮点表示

一、进位计数制及其相互转换

一个数值型数据的完整表示需三个方面：

进位计数制、符号的数字化（机器数）、小数点的处理

1、进位计数制及其相互转换

（二--八--**十**--十六进制间的转换）

1) 构成进位计数制的两个基本要素：

基数、权

基数：每个数位中所允许的最大数码值+1或每个数位中所允许的最多数码个数。

权：是一个与所在数位相关的常数。

权与基数的关系：相邻两位的权值之比等于基数值。

一、数值型数据的表示方法

2) 各种进位计数制的表示 (二--八--十--十六进制)

二进制 (B: Binary system) : 权是 2^i 、基数是0、1

八进制 (O/Q: Octal system) : 权是 8^i 、基数是0、

1、2、3、4、5、6、7

十进制 (D: Decimal system) : 权是 10^i 、基数是0、1、

2、3、4、5、6、7、8、9

十六进制 (H: Hexadecimal system) : 权是 16^i 、基

数是0、1、2、3、4、5、6、7、8、9、A、B、

C、D、E、F

一、数值型数据的表示方法

3) 各种进位计数制的相互表示

(二--八--十--十六进制的转换)

$$(32)_{10} = (100000)_2 = (20)_{16} = (40)_8$$

$$(50)_{10} = (110010)_2 = (32)_{16} = (62)_8$$

$$(110)_{10} = (1101110)_2 = (6E)_{16} = (156)_8$$

例子：十六进制：某机地址总线16位 $A_0 \sim A_{15}$ （高），访存空间64KB。外设与主存采用统一编址，I/O空间占用FC00H $\sim$ FFFFH。问地址总线可访问的主存容量与I/O空间容量分别为多大？（分别为63K、1K）

一、数值型数据的表示方法

2、带符号数的表示

1) 真值：用正、负符号加绝对值表示数值。

2) 机器数、分类及组成

a. 机器数的定义：在计算机内部使用的，连同数符一起数码化了的数。

b. 分类：原码、补码、反码

c. 组成：符号位部分 + 有效数值部分

d. 原码、补码、反码的互换

3、数的定点表示与浮点表示

1) **定点表示法**：在计算机中，小数点位置固定不变的数称为定点数。

表示数的范围和分辨率固定不变。

a. **无符号定点整数**：略去符号位的正整数。

b. **带符号定点整数**：是纯整数，小数点在最低位之后，最高位为符号位。

c. **带符号定点小数**：是纯小数，用原码或补码表示。

一、数值型数据的表示方法

2) 浮点表示法：是一种小数点位置不固定可随需要浮动的数。

a. 浮点数格式

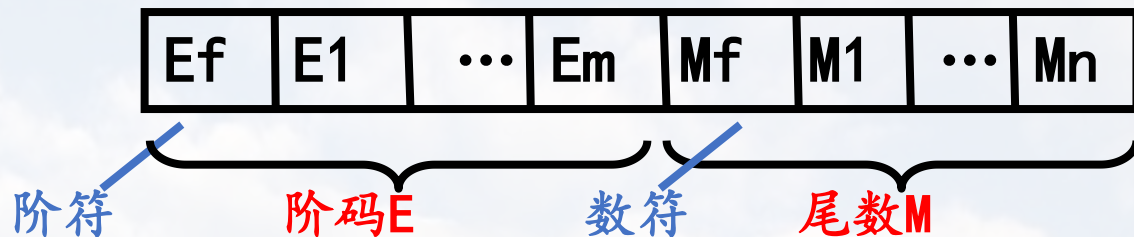
浮点数的代码由两部分组成：阶码E与尾数M。

b. 浮点的真值： $N = \pm R^E \times M$

式中N为真值， R^E 为比例因子，M是尾数。对于某种浮点格式，R固定不变且隐含约定，因此浮点数代码序列中只需分别给出E和M两部分（连同它们的符号）。

一、数值型数据的表示方法

浮点数机器格式：



R 是阶码的底，一般选择与尾数 M 的基数相同， R 是隐含约定的常数，不直接表示在浮点数代码之中。例如尾数为二进制，基数为2，则选择 $R=2$ 。

E 是阶码，也就是比例因子 R^E 的指数值，为二进制正负整数，可用补码或移码表示（有关移码的概念将在后面介绍）。若阶码 E 为正，表明尾数 M 将被扩大若干倍。若 E 为负，表明 M 将被缩小若干倍。

M 是尾数，为带符号定点小数，可用补码或原码表示。

一、数值型数据的表示方法

移码（增码）

前面提到的移码也称为增码，常用于表示浮点数的阶码中。移码定义如下：

若阶码（连同一位符号位）共 $m+1$ 位，代码序列为 $X_m X_{m-1} \cdots X_1 X_0$ ，则

$$X_{\text{移}} = 2^m + X \quad -2^m \leq X < 2^m$$

式中 X 是阶码的真值， 2^m 是符号位 X_m 的位权。因此 $X_{\text{移}}$ 相当于将真值 X 沿数轴正向平移 2^m ，所以称为移码。或者说将 X 增加 2^m ，所以又取名为增码。

一、数值型数据的表示方法

例：若某浮点数阶码（连同一位符号位）共8位，移码表示，表示范围 $-128 \leq X \leq 127$ ，则 $X_{\text{移}} = 2^7 + X$ 。真值、移码、补码对应表如下：

真值X（十进制）	真值X（二进制）	$X_{\text{补}}$	$X_{\text{移}}$
-128	-10000000	10000000	00000000
-127	-01111111	10000001	00000001
...			
-1	-00000001	11111111	01111111
0	00000000	00000000	10000000
+1	00000001	00000001	10000001
...			
+127	01111111	01111111	11111111

一、数值型数据的表示方法

移码的特点如下：

- a. 最高位为符号位，但其取值与原码、补码正好相反。
- b. 除符号位相反之外，移码的其余各位与补码相同。

这是由于移码平移了 2^7 ，而补码则平移了 2^8 （模值）。

- c. 让 X 从 -128 逐渐增至 $+127$ ，相应地 $X_{\text{移}}$ 从 $00\cdots 00$ 逐渐增至 $11\cdots 11$ ，呈递增状。可见采用移码能更直观地比较正负阶码的大小，例如 $+1$ 与 -127 之间的比较。

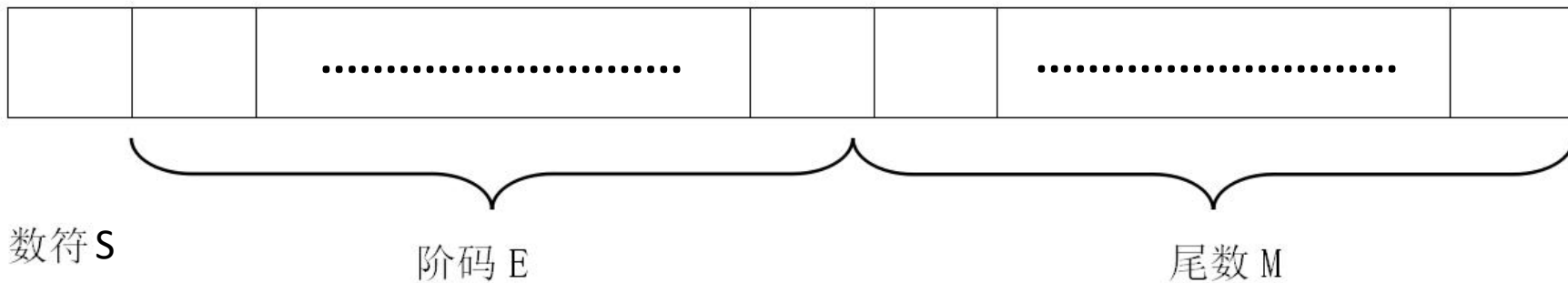
3) 表示范围与精度

一、数值型数据的表示方法

4) IEEE754标准浮点格式

前面讨论的是原理性浮点格式，但实际计算机的浮点格式与此有一些差异。下面简要介绍在当前主流微机中广泛采用的IEEE754标准浮点格式。

按IEEE754标准，常用的浮点数的格式如下图所示：



一、数值型数据的表示方法

IEEE754有3种浮点表示格式，分别称为：**短浮点数**（或称**短实数**、或称**单精度浮点数**）、**长浮点数**（或称**长实数**、或称**双精度浮点数**）、**临时浮点数**（或称**临时实数**、或称**扩展精度浮点数**）。

类型	数符S(位)	阶码E(位)	尾数M(位)	总位数(位)	偏置值	
					十六进制	十进制
短浮点数	1	8	23	32	7FH	127
长浮点数	1	11	52	64	3FFH	1023
临时浮点数	1	15	64	80	3FFFH	16383

短浮点数又称为单精度浮点数，长浮点数又称为双精度浮点数，它们都采用隐含尾数最高数位 (2^0) 的方法，这样，无形中又增加了一位尾数，因此，相应地尾数真值实际上等于 $1 + (\text{23位尾数数值或52位尾数数值})$ 。

临时浮点数又称为扩展精度浮点数，它没有隐含位，尾数真值就等于64位尾数数值。

一、数值型数据的表示方法

下面以32位短浮点数为例，最高位是数符S，其后是8位阶码E，以2为底，采用移码表示，但偏置量为127，例如阶码真值为1，则阶码的代码值为128，这点与前述原理性偏置量（128）有点差异。其余23位尾数为纯小数，因此，尾数位数实际上是：

1位隐含位+23位尾数=24位

注意：

隐含的“1”是一位整数（即权位为 2^0 ）。在浮点格式中表示出来的23位尾数是纯小数，用原码表示。

例如： $(15)_{10} = (1111)_2$ ，将它规格化后结果为 1.111×2^3 ，其中整数部分的“1”将不存储在23位尾数内。

阶码是以移码形式存储的。

短浮点数的偏置值为十进制127或十六进制7FH;

长浮点数的偏置值为十进制1023或十六进制3FFH;

临时浮点数的偏置值为十进制16383或十六进制3FFFH。

一、数值型数据的表示方法



存储浮点数阶码部分之前，偏置值先要加到阶码真值上。

若阶码真值为3，在短浮点数中，移码表示的阶码为：十进制 $127+3=130$ 或十六进制82H；

长浮点数中，移码表示的阶码为：十进制 $1023+3=1026$ 或十六进制402H；

临时浮点数中，移码表示的阶码为：十进制 $16383+3=16386$ 或十六进制4002H。

一、数值型数据的表示方法

方法一：通用法

例：将 $(82.25)_{10}$ 转换成短浮点数格式。

1) 先将 $(82.25)_{10}$ 转换成二进制数

$$(82.25)_{10} = (1010010.01)_2$$

整数部分：除基取余法
小数部分：乘基取整法

2) 规格化二进制数 $(1010010.01)_2$

$$1010010.01 = 1.01001001 \times 2^6$$

3) 计算移码表示的阶码=偏置值+阶码真值：

$$(127+6)_{10} = (133)_{10} = (10000101)_2$$

一、数值型数据的表示方法

4) 以短浮点数格式存储该数

因此：符号位 $S=0$

表示该数为正数

阶码 $E=10000101$

由3) 可得

尾数 $M=01001001, \underline{0000000000000000}$ 由2) 可得

尾数为23位，不足在后面添15位0

所以，32位短浮点数代码为：

$\begin{matrix} S & E & M \\ \downarrow & \downarrow & \downarrow \end{matrix}$
 $0; 10000101; 010010010000000000000000$ (计算机中的实际存储)

表示为十六进制代码为：42A48000H (书写表示)

一、数值型数据的表示方法

例：将 $(-18.125)_{10}$ 转换成短浮点数格式。

1) 先将 $(-18.125)_{10}$ 转换成二进制数

$$(-18.125)_{10} = (-10010.001)_2$$

2) 规格化二进制数 $(-10010.001)_2$

$$-10010.001 = -1.0010001 \times 2^4$$

3) 计算移码表示的阶码=偏置值+阶码真值：

$$(127+4)_{10} = (131)_{10} = (10000011)_2$$

一、数值型数据的表示方法

4) 以短浮点数格式存储该数

因此：符号位 $S=1$

表示该数为负数

阶码 $E=10000011$

由3) 可得

尾数 $M=0010001, \underline{0000000000000000}$ 由2) 可得

尾数为23位，不足在后面添16位0

所以，短浮点数代码为：

$1;10000011;001000100000000000000000$ (计算机中的实际存储)

表示为十六进制代码为： $C1910000H$ (书写表示)

方法二：公式法

单精度浮点计算公式：

$$(-1)^S \times 1.M \times 2^{E-127}$$

$\left\{ \begin{array}{l} S: \text{符号位 (1为负, 0为正)} \\ M: \text{尾数, 表示纯小数} \\ E: \text{阶码} \end{array} \right.$

双精度浮点计算公式：

$$(-1)^S \times 1.M \times 2^{E-1023}$$

一、数值型数据的表示方法

例1: IEEE754单精度浮点数: C0A00000H的十进制值是多少?

$$(C0A00000)_{16} = (1100, 0000, 1010, 0000, 0000, 0000, 0000, 0000)_2$$

; 1, 10000001, 010000000000000000000000

可得, 符号位S是1

阶码位E是10000001 $\rightarrow (10000001)_2 = (129)_{10}$

尾数M是: 010, 0000, 0000, 0000, 0000, 0000

$$\rightarrow (0.010000000000000000000000)_2 = (0.25)_{10}$$

因此, 由公式 $(-1)^S \times 1.M \times 2^{E-127}$ 得

$$(-1)^1 \times 1.25 \times 2^{129-127} = -1.25 \times 4 = -5$$

一、数值型数据的表示方法

例2: 将 $(-18.125)_{10}$ 转换成短浮点数格式?

$$(-18.125)_{10} = (-10010.001)_2 = -1.0010001 \times 2^4$$

$$\rightarrow \begin{cases} S=1 \\ M=0.0010001, \text{ 即 } 0010001 \text{ } \underline{0\cdots\cdots\cdots 0} \text{ (连续16个0)} \\ E=127+4=131, \text{ 即 } (131)_{10} = (10000011)_2 \end{cases}$$

$\rightarrow 1; 10000011; 001000100000000000000000$

如何由十进制数转换成长浮点数格式、临时浮点数格式; 以及又如何由短浮点数格式、长浮点数格式、临时浮点数格式转换成十进制数请读者自己思考。



1.2 字符的表示方法

➤ 01. ASCII 码

➤ 02. 汉字编码简介 (自学为主, 不要求)

非数值型数据：一类是字符及以字符为基础的数据信息，另一类是可以数字化的、范围极其广泛的如逻辑信息、图形、图像、声音等等静态的或动态的信息。

本节内容是介绍字符型信息的表示方法。字符是非数值型信息的表示基础，也可以用它来间接描述范围广泛的绝大多数信息。

国际上广泛采用美国信息交换标准码（American Standard Code For Information Interchange）作为标准，简称为**ASCII码**。

在ASCII字符集中共有**128种常用字符**，每个ASCII字符自身有七位编码，存储器中的一个字节单元正好可以存放一个ASCII字符编码与一位奇偶校验位。这样，存储器也就常以字节为一个存储单元的编址单元。

一、ASCII码



十六进制码	字符	十六进制码	字符	十六进制码	字符	十六进制码	字符
00	NUL	20	SP	40	@	60	`
01	SOH	21	!	41	A	61	a
02	STX	22	”	42	B	62	b
03	ETX	23	#	43	C	63	c
04	EOT	24	\$	44	D	64	d
05	ENQ	25	%	45	E	65	e
06	ACK	26	&	46	F	66	f
07	BEL	27	'	47	G	67	g
08	BS	28	(	48	H	68	h
09	HT	29	)	49	I	69	i
0A	LF	2A	*	4A	J	6A	j
0B	VT	2B	+	4B	K	6B	k
0C	FF	2C	,	4C	L	6C	l
0D	CR	2D	-	4D	M	6D	m
0E	SO	2E	.	4E	N	6E	n
0F	SI	2F	/	4F	O	6F	o
10	DLE	30	0	50	P	70	p
11	DC1	31	1	51	Q	71	q
12	DC2	32	2	52	R	72	r
13	DC3	33	3	53	S	73	s
14	DC4	34	4	54	T	74	t
15	NAK	35	5	55	U	75	u
16	SYN	36	6	56	V	76	v

一、ASCII码



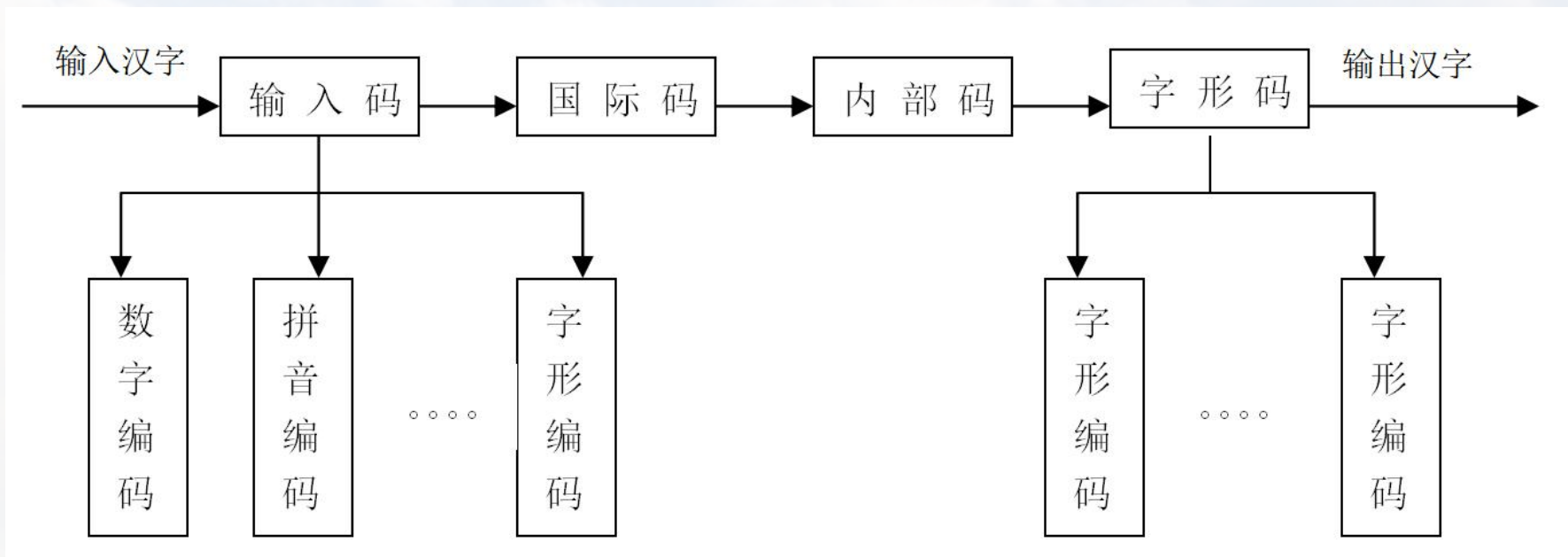
十六进制码	字符	十六进制码	字符	十六进制码	字符	十六进制码	字符
17	ETB	37	7	57	W	77	w
18	CAN	38	8	58	X	78	x
19	EM	39	9	59	Y	79	y
1A	SUB	3A	:	5A	Z	7A	z
1B	ESC	3B	;	5B	[	7B	{
1C	FS	3C	<	5C	\	7C	
1D	GS	3D	=	5D	]	7D	}
1E	RS	3E	>	5E	↑	7E	~
1F	US	3F	?	5F	-	7F	DEL

二、汉字编码简介



汉字也是字符，是中文的基本组成单位。由于汉字数量极大，接近7万个，字形复杂、异体字多、同音字多，因此，汉字信息的处理比西文复杂得多。汉字信息的处理一般包括汉字的编码、输入、输出、存储、处理与传输。

汉字信息处理的模型



本小节主要讨论编码问题，不考虑输入、输出存储、处理与传输。在一个汉字信息处理系统的不同部位，需使用下述三类编码：输入码、内部码、交换码。

1、汉字输入码

每个汉字用一个或几个键输入的编码来表示。这种编码方式称为汉字的输入编码，也称**外码**。

目前汉字的编码方案有几百种之多，较常使用的也有几十种之多。归纳起来，所采用的方法可分为数字码、拼音码、字形码、音形结合、以及具有某些提示和联想功能的编码等几类。

2、汉字内部码

汉字内部码也称**机内码**，它是计算机内部供存储、处理、传输用的代码，简称内码，是汉字在设备或信息处理系统内部最基本的表达。目前国内采用的内部码有四十种左右。

3、汉字交换码

如前所述，早期的各种汉字系统的内码不统一，因此在各汉字系统之间或汉字系统与通信系统之间进行汉字信息交换（即传输）时，需要制定一种编码标准，即汉字交换码。

我国制定了《信息处理交换用的七位编码字符集》，后来成为国家标准GB1988。除个别字符（如货币符号）外，GB1988与ASCII是一致的，可视为ASCII的中国版本。

二、汉字编码简介



GB2312只收录了6763个汉字字符和682个非汉字图形字符（如间隔、标点、运算符、制表符、数字、汉语拼音、拉丁文字母、希腊文字母、俄文字母、日文假名等），已不能满足用户应用的需要。

2000年底又颁布了**GB18030**大字符集标准，该标准涵盖27484个汉字，简繁体字均处于同一平台，并在统一的编码框架下，为以后的扩充提供了充足的空间。

GB18030采用单字节、双字节、四字节混合编码，总编码空间超过150万个。

目前内码推荐方案还不完全等同于汉字交换码国标，但二者之间存在简单的对应关系。



谢谢观看

计算机基础知识

2025-8-22



信息与软件工程学院
School of Information and Software Engineering